

Shortest Job First (SJF)

SJF chooses the shortest ready job whenever the CPU becomes free. Once chosen, the job runs to completion.

1. Model

The visualizer uses one CPU, whole ticks, and CPU-only processes. **Service time** is the total CPU work a process needs. The scheduler knows this value from the input. Running for one tick reduces remaining service by one; switching processes takes no time.

Only processes that have arrived are eligible. A ready process is waiting for the CPU; a future process has not arrived yet. A process's arrival time does not change while it waits. Queue lists show insertion order, with the oldest queued entry on the left. The short examples under the rules are separate scenarios.

2. Scheduling rules

Rule 1. Choose the shortest ready service time

When the CPU is free, compare the original service times of all ready processes and select the smallest. The choice is made from the whole ready queue.

Explanation. SJF uses the CPU work specified in the input. The ready queue is displayed in insertion order, but selection searches the whole queue. The chosen process may therefore come from the middle or back.

Example. Before dispatch, the queue is [B, C]. B needs four ticks and C needs one. Select C and leave B waiting. Waiting longer does not give B priority over a shorter ready job.

Rule 2. Resolve equal lengths by arrival, then input order

Among jobs with equal service times, choose the earlier arrival. If arrival times also match, choose the one listed first in the input. Process IDs do not determine the order.

Explanation. Service time is the first comparison. Arrival and input order are used only when service times match. The letters in a process ID are labels, not priorities.

Example. At time 4, the CPU becomes free. C arrived at time 1 and B at time 2; each needs two ticks. Select C. If both had arrived at time 1, their input order would decide instead: listing B before C would select B.

Rule 3. Do not interrupt a running process

SJF is non-preemptive. A shorter process arriving later must wait for the current one to finish.

Explanation. The shortest-job comparison happens when the CPU is free. A new arrival joins the ready queue without causing another selection while the current process is running.

Example. A starts at time 0 with five service ticks. B arrives at time 1 needing one tick. A still runs from 0 to 5; B runs from 5 to 6. B is shorter, but SJF does not preempt A.

Rule 4. Finish at zero remaining service

A process leaves after its final CPU tick. Dispatch may occur at the same boundary, with no extra tick for the switch.

Explanation. Remove the finished process, admit arrivals at that time, and then compare the ready jobs. A new arrival at the completion boundary is eligible immediately.

Example. A finishes at time 5. B is waiting with four service ticks, and C arrives at time 5 needing one. Select C at time 5. B's earlier arrival does not win because their service times differ.

Rule 5. Do not wait for a future short job

If any process is ready and the CPU is free, select a ready process now. Idle only when none is ready. Stop when all processes finish.

Explanation. Knowing a future process's service time does not make it ready. SJF chooses among the processes available now. It does not deliberately leave the CPU idle to wait for a better candidate.

Example. A arrives at time 3 needing two ticks; B arrives at time 4 needing one. The CPU is idle before time 3. When A arrives, it starts immediately and runs until time 5. B then runs until time 6.

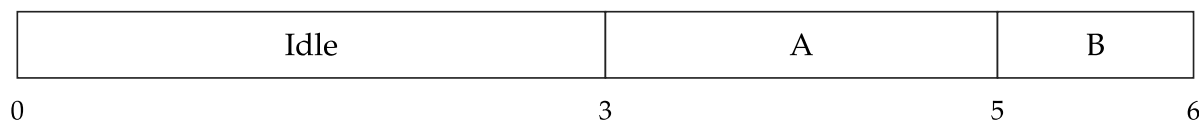


Figure 1. CPU schedule (time in ticks).

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. Admit all arrivals at the current time.
3. If the CPU is free, choose the shortest ready job, using Rule 2 for ties.

Rule 4's example shows why arrivals are admitted before selection. If the running process has not finished, Rule 3 keeps it on the CPU regardless of those arrivals.

4. Worked example

Input order is A, B, C. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

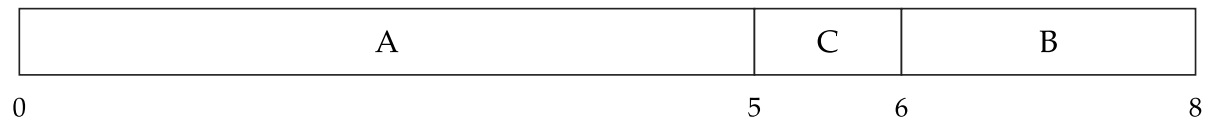


Figure 2. CPU schedule (time in ticks).

A is the only ready process at time 0, so it starts immediately. B and C arrive at time 1 and wait. At time 5, C is chosen because its one-tick service is shorter than B's two-tick service. B starts after C finishes at time 6.